

1. Overview

Objective: Compute deterministic subproblems before language generation (CGR).

Limit: Answer-only scores cannot distinguish grounded geometry from guessing.

KEY TAKEAWAY

Computing what can be computed before generating what must be generated.

2. Spatial Atlas System

Two benchmark-oriented handlers share one A2A server, model routing, budgets, and provenance controls.

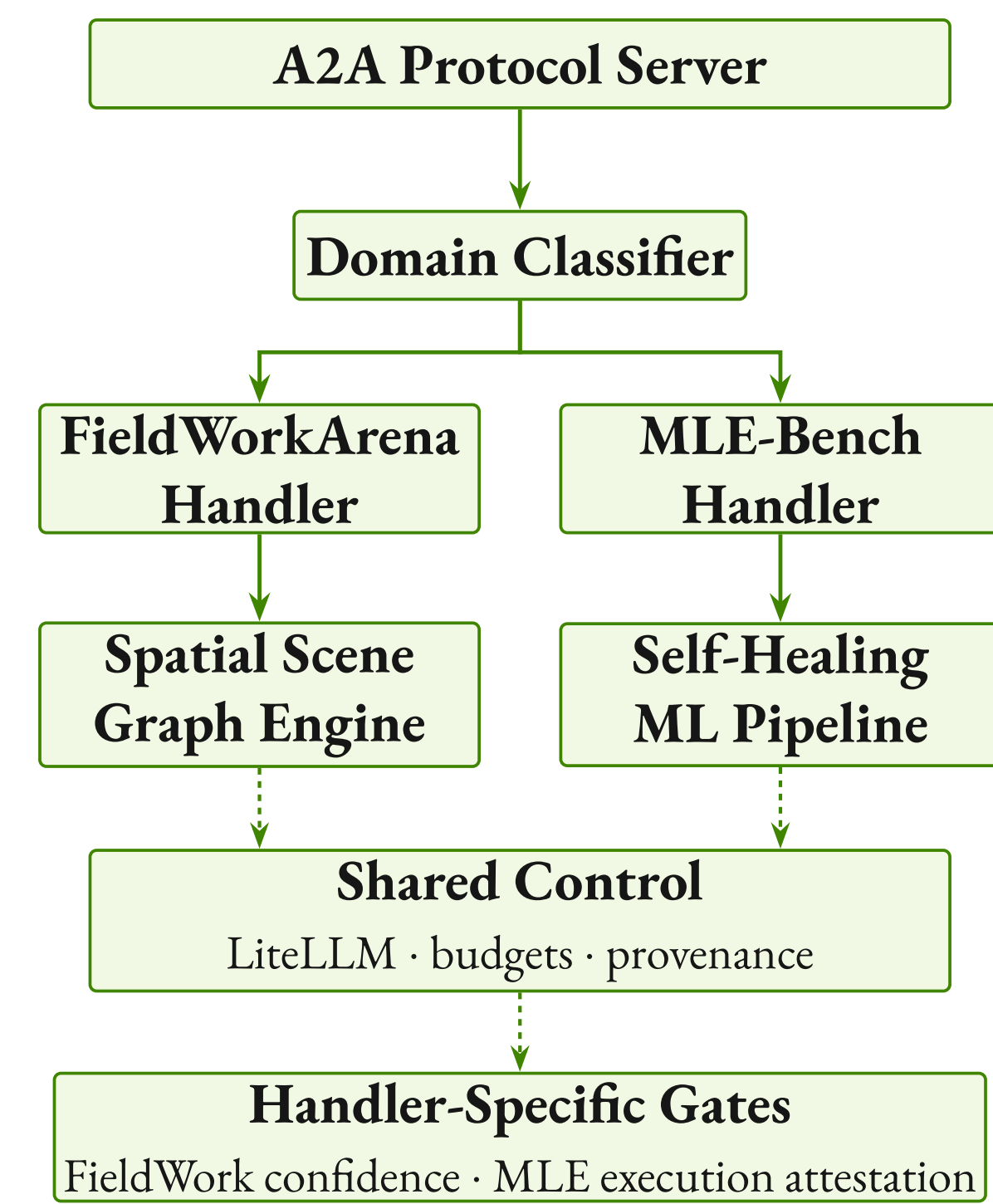


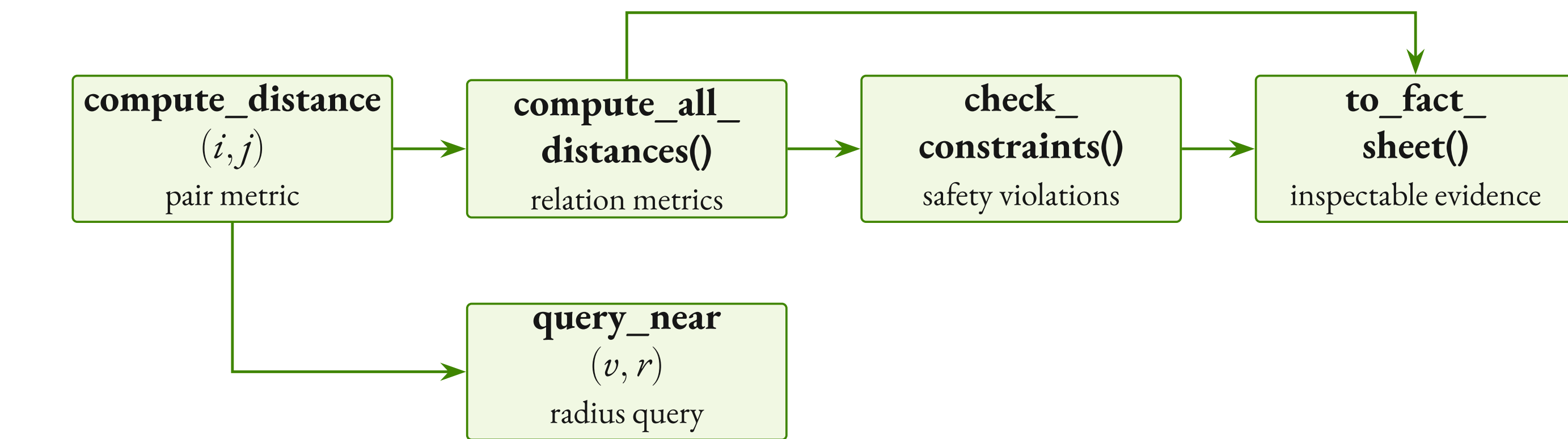
Figure 1. Spatial Atlas architecture. A classifier routes tasks to two handlers that use shared control services and apply handler-specific gates. Dashed lines indicate shared services.

3. Contributions

Computable evidence substrate: Given an image I and question q , CGR separates evidence construction from answer generation by creating a typed graph $G = (V, E)$:

$v_i = \text{SpatialEntity}(\text{id}_i, \ell_i, \text{pos}_i, \text{attrs}_i, z_i)$ (1) $e_{ij} = \text{SpatialRelation}(i, p_{ij}, j, d_{ij})$ (2)

Deterministic graph program: The graph exposes four computation and query methods plus one fact-sheet serializer over vision-language-estimated coordinates, producing structured evidence rather than measured 3D geometry.



SCENE-GRAPH DATAFLOW DAG

- **Scene-graph flow:** Extraction proposes entities and approximate positions $\rightarrow$ structuring writes typed vertices and relations $\rightarrow$ computation evaluates geometry and constraints on the fixed graph. `compute_distance(i, j)` feeds the bulk path `compute_all_distances()` $\rightarrow$ `check_constraints()` $\rightarrow$ `to_fact_sheet()`, which serializes stored distances and violations. It also feeds the independent read-only `query_near(v, r)` radius branch, which is not required for serialization.
- **Entropy-Guided Reasoning:** At step t , the knowledge state $\mathcal{K}_t$ aggregates observations, computed facts, and intermediate conclusions. The paper defines answer entropy and selects the action with the greatest expected entropy reduction:

$$H(\mathcal{A} \mid \mathcal{K}_t) = - \sum_{a \in \mathcal{A}} P(a \mid \mathcal{K}_t) \log P(a \mid \mathcal{K}_t) \quad (3)$$

$$c^* = \arg \max_{c_j} \mathbb{E}[H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \text{obs}(c_j))] \quad (4) \quad \text{reflect}(a) = \begin{cases} \text{True}, & \sigma(a) < \tau \\ \text{False}, & \text{otherwise} \end{cases} \quad (5)$$

Approach and why it will succeed: Four mechanisms instantiate compute-grounded reasoning (CGR) end to end.

- **Typed scene evidence:** entities, estimated positions, relations, zones, and constraints.
- **Confidence-gated reflection:** one conditional refinement when self-graded confidence is below 0.6.
- **Fail-closed execution:** generated code requires explicit opt-in and isolated-worker attestation.
- **Repair then score-gated refinement:** diagnose and patch failures using error and stdout traces. After successful execution, retain a refinement only when its parsed validation score improves.

The spatial branch computes relations and facts. The ML branch analyzes tasks, generates and repairs pipelines, and compares pipeline-emitted validation scores.

The proposed routing policy returns a Fast-tier answer above 0.8, invokes Standard from 0.6 through 0.8, and reflects or escalates below $\tau = 0.6$, for at most two rounds. This policy is not the deployed FieldWorkArena path. The deployed controller neither evaluates the information-gain objective nor demonstrates calibration of its self-grade.

Require: task T , state $\mathcal{K}_0$, budget B , threshold τ

```

1  $a_0, \sigma_0 \leftarrow \text{FastModel}(T, \mathcal{K}_0)$ 
2 if  $\sigma_0 \geq 0.8$  then
3   return  $a_0$ 
4 end if
5  $\mathcal{K}_1 \leftarrow \mathcal{K}_0 \cup \text{SceneGraph}(T)$ 
6  $a_1, \sigma_1 \leftarrow \text{StandardModel}(T, \mathcal{K}_1)$ 
7 for  $r = 1$  to 2 do
8   if  $\sigma_1 \geq \tau$  then
9     return  $a_1$ 
10  end if
11   $\mathcal{K}_{r+1} \leftarrow \text{Reflect}(\mathcal{K}_r, a_1)$ 
12   $a_1, \sigma_1 \leftarrow \text{StrongModel}(T, \mathcal{K}_{r+1})$ 
13 end for
14 return  $a_1$ 

```

4. Evaluation

FieldWorkArena Handler: The v2 preprint describes a pipeline using Florence-2, GPT-4.1 vision, and bounding-box-centroid positions. The current repository instead obtains `position_x` and `position_y` from a Strong-model JSON extraction prompt.

- 1 Describe the image and extract entities and relations.
- 2 Store a typed scene graph $G = (V, E)$.
- 3 Compute pairwise distances and safety-rule violations.
- 4 Pass a fact sheet to the final language model.

MLE-Bench Handler: Execution of generated code follows a fail-closed policy that requires explicit operator opt-in and isolated-worker attestation.

- 1 Analyze task type, metric, files, and constraints.
- 2 Generate a strategy-specific Python pipeline.
- 3 Run only inside the bounded worker after the gate.
- 4 Diagnose, patch, and retry failures for at most 3 attempts.
- 5 Parse VALIDATION_SCORE after success.
- 6 Refine at most twice and retain only a score improvement.

The rollback rule is explicit:

$$p_{t+1} = \begin{cases} p' & \text{better}(s(p'), s(p_t)) \\ p_t & \text{otherwise} \end{cases} \quad (6)$$

Prompted safeguards: The generated pipeline is instructed to check ID overlap, row fingerprints, temporal order, and byte-identical media. These checks are not independently enforced. Dummy fallback remains opt-in.

5. Source-Reported Results

Configuration	Factory	Warehouse	Retail
w/o Spatial Scene Graph	51	44	55
w/o Entropy-Guided Reasoning	65	60	67
w/o Florence-2	63	58	66
GPT-4V Direct	48	41	52
Overall	72	68	74

REPORTED RESULT

Removing the Spatial Scene Graph changes Factory, Warehouse, and Retail accuracy by -21 , -24 , and -19 points. MLE-Bench reports **82% valid submissions** and **32% medals** across 75 competitions.

Limitation without SpatialClaw: Atlas computes deterministic relations over model-estimated 2D coordinates. Localization and depth errors can therefore propagate into distances and constraints.

Where SpatialClaw works best: Multi-step tasks with reliable masks, depth, or point maps that require geometric inspection, operation composition, evidence recomputation, and answer revision.



6. SpatialClaw Integration

Integration design: Spatial Atlas specifies the evidence that should be available before generation. SpatialClaw contributes a stateful code-action interface that can inspect tool outputs, compose operations, and revise computation after each observation.

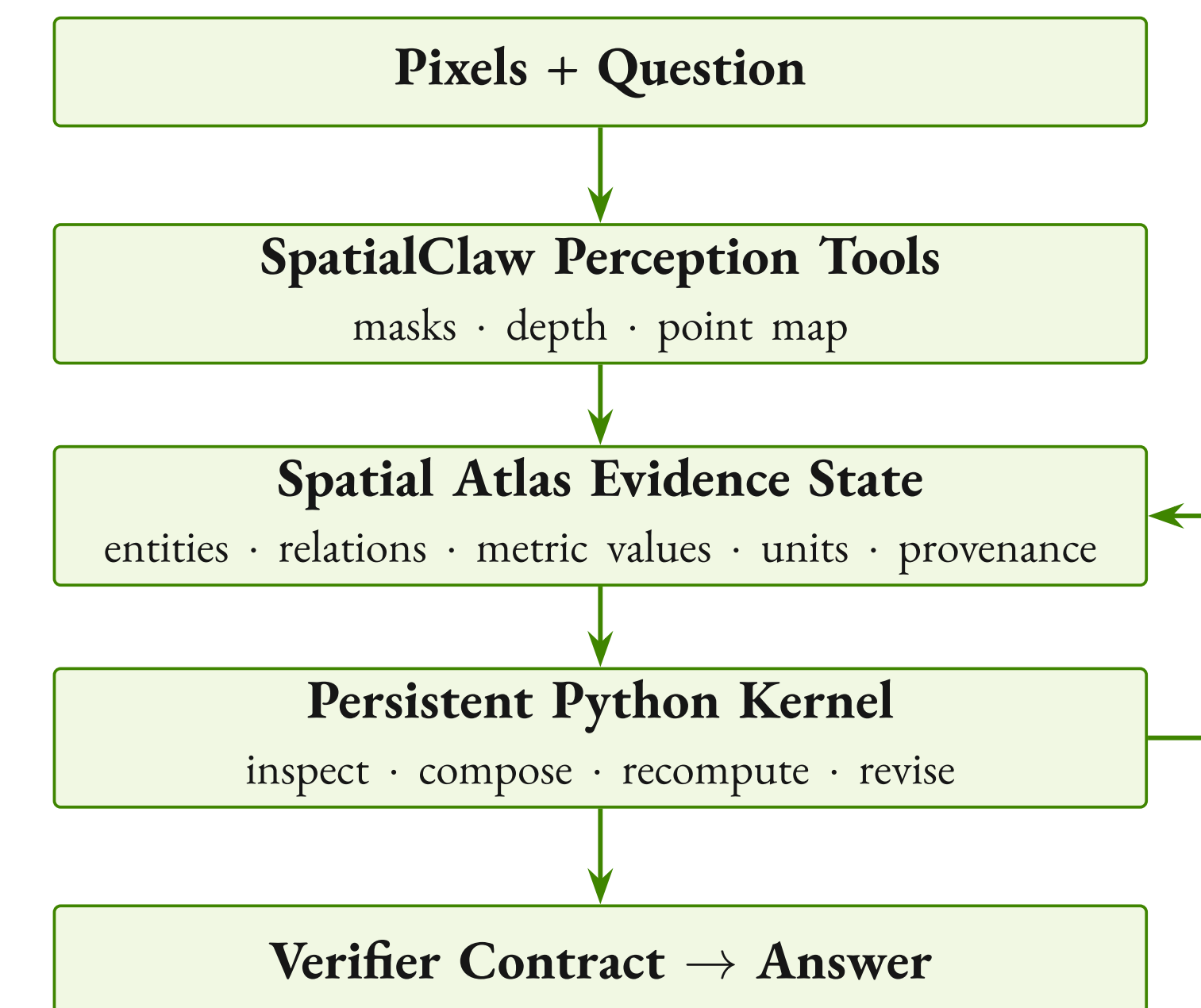


Figure 2. Integration design. The local metric bridge is implemented. The persistent-kernel link, evidence journal, and post-kernel verifier are the next integration steps.

UPSTREAM CONTEXT

SpatialClaw is a separate, training-free framework that treats code as the action interface. A VLM writes one AST-checked Python cell per step in a persistent kernel, then conditions later actions on text, variables, errors, and rendered evidence. The kernel exposes SAM3 and Depth-Anything-3 perception primitives with scientific libraries. The paper reports **59.9% average accuracy across 20 spatial reasoning benchmarks** and **+11.2 points** over the prior best spatial agent, using the same prompt, tool set, and hyperparameters across six VLM backbones. These are SpatialClaw results, not Spatial Atlas evaluations. Its tasks, metrics, models, and perception stack differ.

7. Integration Contributions

7.1 Measured Geometry Bridge

The FieldWork handler can switch from the scene graph to a metric backend that invokes SpatialClaw perception primitives. Strict QSpatial mode fails closed rather than falling back.

- 1 Obtain object masks and a reconstructed point map.
- 2 Erode masks and retain finite, confidence-qualified points.
- 3 Build deterministic horizontal XZ voxel representatives.
- 4 Compute symmetric directed nearest-neighbor 5th percentiles.
- 5 Validate metric range, provenance, protocol contracts, and evidence hashes.

$$d_{A \rightarrow B} = Q_{0.5} \left(\left\{ \min_{b \in B} \|a - b\|_{XZ} : a \in A \right\} \right) \quad (7)$$

$$\hat{d}(A, B) = \min(d_{A \rightarrow B}, d_{B \rightarrow A}) \quad (8)$$

SpatialClaw supplies the perception primitives. Erosion, voxelization, the surface-gap estimator, strict validation, and the Atlas scene bridge are local Spatial Atlas work.

7.2 Shared Action Contracts

The integration contract defines four requirements:

- **Kernel-visible state:** expose entities, values, confidence, units, and provenance.
- **Stateful contract:** inspect, compose, recompute, and revise evidence per observation.
- **Evidence-use journal:** record facts and tool outputs used by each answer.
- **Verifier contract:** count only answers backed by fresh, valid, attested evidence.

TAKE-AWAY

Spatial Atlas makes evidence explicit and verifiable. SpatialClaw adds metric perception and a stateful action loop so spatial answers can be recomputed, revised, and traced to evidence.